vasic-digital tmux — optimized + verified containerized build

What's new in v1.0.15 / v1.0.16 (2026-05-28)

- **Multi-line copy + paste-IN + Claude Code TUI support.** New `bind -n M-MouseDrag1Pane copy-mode -M` (Alt-drag, macOS) and `bind -n S-MouseDrag1Pane copy-mode -M` (Shift-drag, Linux) force tmux selection even when Claude Code / vim / less / htop hold the alt-screen with mouse tracking. New `@clip-read` user option + `prefix + P` paste-IN binding pulls the OS clipboard INTO the current pane through `tmux paste-buffer -p` (bracketed paste). See docs/guides/clipboard.md for the operator recipe.
- **Workable-items SQLite SSoT (`cmd/workable-items/`).** Project-local Go binary implementing the §11.4.93 SQLite single-source-of- truth for every workable item in `Issues.md` / `Fixed.md`. Pure-Go (no CGO) via `modernc.org/sqlite`. DB at `docs/workable_items.db` is **TRACKED in git per §11.4.95**. See docs/workable-items/README.md for the subcommand reference.
- **DOCX export on every Markdown sync.** `scripts/sync_all_markdown_exports.sh` now produces `.docx` siblings alongside the existing `.html` + `.pdf` exports — verified across 44 candidate documents with pandoc-emitted "Microsoft Word 2007+" output.

A reproducible, hardened build of tmux with built-in jemalloc support, OOM-protection helper, and a comprehensive verification gate. **Runs natively on any Linux host** (Ubuntu, ALT, Fedora, Arch, openSUSE, Alpine) where podman or docker is available. **macOS hosts** (Apple Silicon + Intel) are supported via a transparent bridge into the podman machine VM — the operator gets a working `tmx` command on the macOS shell with no manual SSH-juggling.

The 18 verification tests are why this matters: a typical "build tmux from source" guide assumes the build worked. This project ships a hard wall — `bash scripts/setup.sh` will refuse to PATH-export the binary unless functional tests pass with positive

runtime evidence (cgroup interface readbacks, `/proc` files, real session output), backed by a §11.4.4 layer-4 paired-mutation harness that proves the gates aren't themselves bluffs. SKIPs document precondition gates (CAP_SYS_RESOURCE, libjemalloc presence, destructive-test opt-in) explicitly. No PASS-bluffs.

Install (one-liner)

Obtain and run the installer in a single `curl` command, like any modern CLI:

```
curl -fsSL https://raw.githubusercontent.com/vasic-digital/tmux/
        main/scripts/install.sh | bash
```

That one command **clones the whole project (with all submodules, fully recursive) → builds + verifies (`scripts/setup.sh`) → runs the full validation suite (`scripts/tests/run_all.sh`) → wires `tmx` onto your PATH**. It installs into `$HOME/tmux` by default (the project name, lowercase snake_case per the constitution naming convention). When it finishes, `source ~/.bashrc` (or `~/.zshrc`) — or open a new terminal — and run `tmx new -s <name>`.

It is honest by construction (§11.4 anti-bluff): any failure in clone, build, verification, or tests makes the installer **exit non-zero** — it never PATH-exports an unverified binary, and it **refuses to clobber** a non-empty directory that is not our checkout (§9.2). It runs **no `sudo`** under the pipe.

Prerequisites: `git` (required), plus either a **C toolchain** (compiler + `libevent-dev` + `libncurses-dev`) **or** a **container engine** (podman/docker) for the build — `setup.sh` picks the right path and surfaces missing deps honestly. On Linux, install build deps once with `sudo bash scripts/install_deps.sh`. Missing runtime deps (jemalloc) are obtained git-ignored into `.local-deps/` automatically (§11.4.77).

Options (env var OR flag — flag wins; pass flags under the pipe with `bash -s -- …`):

Env var	Flag	Default	Meaning
`TMX_INSTALL_DIR`	`--dir DIR`	`$HOME/tmux`	install root
`TMX_REPO_URL`	`--repo URL`	`https://github.com/vasic-digital/tmux.git`	clone source (HTTPS so keyless users can clone)
`TMX_INSTALL_BRANCH`		`main`	branch to clone / track

Env var	Flag	Default	Meaning
	`--branch B`		
`TMX_INSTALL_NO_SETUP=1`	`--clone-only`	(off)	clone + submodules only (no build / no host writes)

Re-running the installer over an existing checkout **updates** it (`git pull --ff-only` + recursive submodule update) — idempotent.

File exports the installer wires (what ends up on your host): the PATH+session snippet is appended to whichever shell rc your host uses — **`~/.bashrc`** and/or **`~/.zshrc`** (and `~/.bash_profile` / `~/.profile` for bash login shells); **`~/.tmux.conf`** is installed (any pre-existing non-ours config is backed up to `~/.tmux.conf.pre-vasic-digital`); and the generated **`scripts/tmx`** wrapper is prepended onto PATH so `tmx` resolves to this verified build while the system `tmux` stays reachable side-by-side. Full reference: `docs/scripts/install.md`.

Quick install (one command)

Linux host:

```
git clone --recurse-submodules git@github.com:vasic-digital/
        tmux.git ~/Projects/tmux
cd ~/Projects/tmux
sudo bash scripts/install_deps.sh    # one-time host build deps
bash scripts/setup.sh                 # build + verify + install
        (no sudo)
```

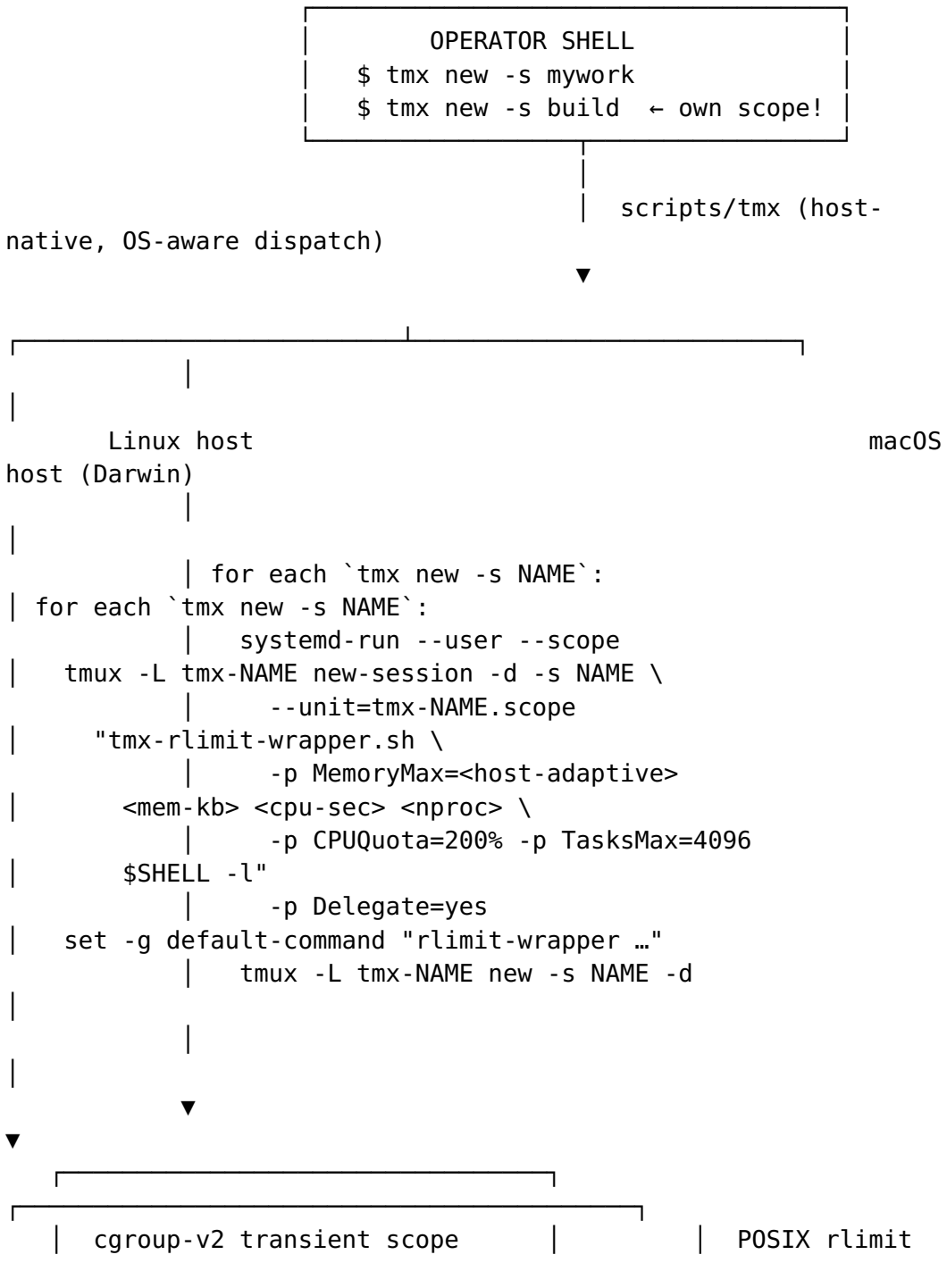
macOS host (Apple Silicon or Intel):

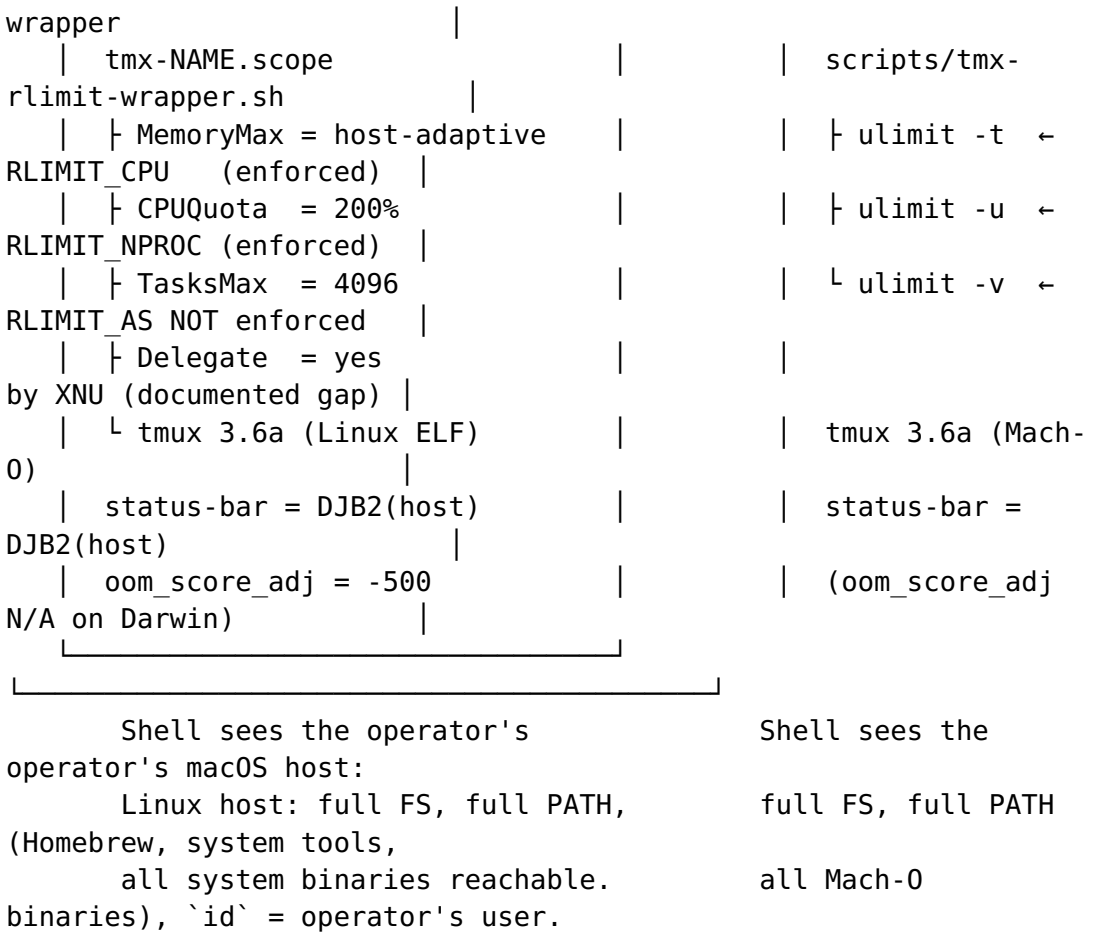
```
brew install podman                   # one-time: container
        runtime
podman machine init && podman machine start
git clone --recurse-submodules git@github.com:vasic-digital/
        tmux.git ~/Projects/tmux
cd ~/Projects/tmux
bash scripts/setup.sh                 # build + VM-verify +
        install bridge
```

After `setup.sh` reports GREEN: open a new shell, or source the rc for your shell (`source ~/.bashrc` or `source ~/.zshrc`). Then `tmx new|attach|ls|kill` invokes the verified vasic-digital build; the system `tmux` command stays untouched and reachable side-by-side.

> **Local dependencies (§11.4.77).** Missing build/runtime deps (e.g. jemalloc) are obtained git-ignored into `.local-deps/` automatically by setup via `scripts/obtain_local_deps.sh` — it resolves a present dep by absolute path (§11.4.111; fixes the non-interactive-PATH case) and builds/extracts it locally when missing (cross-platform: Linux source/container, macOS source/brew). See docs/scripts/obtain_local_deps.md.

Architecture (native dual-OS per-session isolation)





Native dual-OS per-session isolation (architecture since 2026-05-13). Each `tmx new -s NAME` invocation creates its own tmux server (socket `tmx-NAME`) with OS-native isolation:

- **Linux** — cgroup-v2 transient scope `tmx-NAME.scope` via `systemd-run --user --scope`. Kernel enforces MemoryMax, CPUQuota, TasksMax per-group. OOM in one session is contained to that scope — every other session AND `user.slice` survive (Constitution §1 invariant).
- **macOS (Darwin)** — POSIX rlimit wrapper sets `RLIMIT_CPU` (CPU time) and `RLIMIT_NPROC` (per-user process count) before exec'ing the operator's `$SHELL`. The Darwin XNU kernel enforces these per-process. Children inherit. **`RLIMIT_AS` (virtual memory) is NOT enforced** by XNU for unprivileged processes — this is a documented gap; full memory containment on macOS requires launchd jobs with `HardResourceLimits` plist (root). See `docs/guide/README.md` §5.6 for the strength comparison.

Both OS paths deliver the **same operator UX**: plain-vanilla tmux behaviour, the operator's shell with full host PATH (Homebrew on macOS, /usr/local/bin on Linux, all system tools), per-session resource ceilings applied transparently. No VM. No bridge. No `core@localhost`. No bluff.

What you get

Component	Why
tmux 3.6a (latest stable)	Pinned to a known-good upstream tag
Hardened compile flags	`-O2 -DNDEBUG -fstack-protector-strong -D_FORTIFY_SOURCE=2`, RELRO + immediate-binding link
Build-time `-ljemalloc`	jemalloc linked at the binary level (more aggressive RAM return than glibc malloc)
Runtime `LD_PRELOAD=libjemalloc.so`	Wrapper preloads jemalloc even on hosts where the linker resolved a different malloc
OOM-score protection	Optional setcap-enabled helper (`tmx-oom-set`) sets `oom_score_adj=-500` on the spawned server, making tmux survive most OOM cascades
Bounded `history-limit`	Explicit `2000` (the default — explicit so future bumps are intentional)
Per-session cwd memory (v1.0.13+)	`tmx-shell-init.sh` installs a `PROMPT_COMMAND` / `precmd` hook inside every pane; every command's cwd is recorded to `~/.tmx/state.json`. Reopen the same session name → wrapper passes `-c <last-pwd>` to `tmux new-session`; the pane materialises where you left off. End-to-end guarantee verified by `scripts/tests/43_e2e_cwd_persist_real_shell.sh`.
Hermetic install	Built artifact lives in `tmux/build/`. PATH export points there; system tmux untouched. Removable via `bash scripts/uninstall.sh` (or `bash scripts/setup.sh --uninstall` — both delegate to the same single source of truth).

Verification gate

```
SUMMARY: PASS=41  FAIL=0  SKIP=3
GREEN: tmux binary verified — safe to PATH-export.
```

The 44 numbered tests cover (among others): smoke + binary version, session lifecycle, jemalloc loaded, history-limit honored, clear-history releases memory, 10 concurrent panes, sustained

session no-leak, OOM-score wrapper, crash isolation scope (cgroup-v2 transient), hostname-derived status-bar colour (algorithm + integration), memory pressure under cap, TasksMax stress, concurrent OOM independence, per-session cgroup distinctness, window-name `.exe` strip, **scrollback + copy-mode scrolling** (operator-path: 3000 lines generated, proven scrolled off-screen, reachable via copy-mode), HelixConstitution inheritance (submodule + every governance doc's pointer), CodeGraph index materialisation, cross-platform parity branches (macOS ↔ Linux), tmx-shell-init non-TTY guard, tmx-state cwd persistence end-to-end across exit + reopen, SSH dispatch to remote nezha, dispatcher session-name validation, setup install/uninstall E2E, and — new in **v1.0.14 — clipboard copy-OUT physically proven** end-to-end (test 44: marker round-trip through `y` keystroke → `@clip` shell-pipe → `pbpaste` / `wl-paste` / `xclip` / `termux-clipboard-get` returns the marker).

Four honest SKIPs document precondition gates:

- `03_jemalloc_loaded` SKIPs if host doesn't have libjemalloc — `sudo bash scripts/install_deps.sh` provides it
- `08_oom_score_adj` SKIPs unless running as root OR the setcap helper is installed — `sudo bash scripts/build_oom_set.sh --install` enables it
- Tests 12 / 13 / 14 (memory-pressure / TasksMax / concurrent-OOM) require `TMX_TEST_DESTRUCTIVE=1` — run only on dedicated test hosts

A layer-4 paired-mutation harness (Constitution §103) lives at `scripts/tests/meta_test_false_positive_proof.sh`: registered mutations M1–M14 plus `CM-CONSTITUTION-INHERITANCE` each break a feature, assert the matching test then FAILs, revert, and assert it PASSes again. On macOS the harness reports **18 caught / 0 escaped / 6 skipped** (the 6 SKIPs are Linux-only isolation mutations); the remainder run on Linux via `META=1 bash scripts/test_vm.sh`. The gate is not considered self-validating until every runnable mutation is caught.

Roadmap

See docs/plans/containerization.md for the **per-session containerization plan** — each tmux session running in its own cgroup-bounded container so that:

- 2 CPU + reasonable RAM cap per session
- Crash isolation: if one session OOMs/crashes, only that session dies; other sessions and their processes survive
- One-command bootstrap: `tmx new <session>` transparently creates the container

Documentation map

Every project doc lives under a context-named subdirectory of `docs/` per `constitution/Constitution.md`'s file-layout rule. Every Markdown has a synced HTML + PDF sibling generated by `scripts/export_docs.sh` per §11.4.65 universal-Markdown-export.

Area	Doc
Operator guide	`docs/guide/README.md` — install, OS-by-OS notes, isolation comparison, troubleshooting
v1.0.14 / v1.0.15 clipboard	`docs/guides/clipboard.md` — multi-line copy + paste-IN + Alt/Shift-drag inside Claude Code
v1.0.15 workable-items SSoT	`docs/workable-items/README.md` — `cmd/workable-items/` Go binary, subcommand reference, honest gaps
v1.0.9 shell-session resume	`docs/manual/tmx-shell-integration.md` — end-user master manual with copy-paste worked examples
v1.0.9 shell integration	`docs/guides/tmx-shell-integration.md` — operator install/uninstall guide for `tmx-shell-init.sh`
v1.0.9 state daemon	`docs/guides/tmx-state.md` — operator CLI reference + state-file schema for `tmx-state`
v1.0.9 SSH dispatch	`docs/guides/tmx-ssh-dispatch.md` — `ssh <host>-tmx <session>` install, security, troubleshooting
Scrolling (Claude Code TUI + mobile)	`docs/scrolling/README.md`
CodeGraph (§11.4.78)	`docs/codegraph/README.md` — install, per-agent MCP wiring, anti-bluff verification
Local dependencies (§11.4.77)	`docs/scripts/obtain_local_deps.md` — per-host obtain-or-resolve mechanism for git-ignored deps (jemalloc)
Architecture plans	`docs/plans/native-dual-os.md` — current native-host design (no VM)
	`docs/plans/per-session-isolation.md` — per-session OOM containment
	`docs/plans/containerization.md` — original (now superseded) containerization plan
Cycle plans	`docs/plans/v1.0.4.md` — this cycle's plan (CodeGraph + covenant propagation + AUDIT fixes)
Research notes	`docs/research/customization/colors.md` — hostname-derived status colour

Area	Doc
Governance	Constitution.md — Project Articles §101–§109 (extends constitution/); CLAUDE.md, AGENTS.md, QWEN.md — per-agent inheritance pointers + project-specific overlay
Universal governance	constitution/ submodule (HelixDevelopment/HelixConstitution, pinned 7f738df)
Live state	CONTINUATION.md — read first on a fresh conversation
Tracker	Issues.md (open) / Fixed.md (closed with closure SHA + evidence)
Changelog	CHANGELOG.md — per-release positive-evidence verification record

Repository conventions

This repo follows the **vasic-digital anti-bluff covenant**: every test that PASSes carries positive evidence of the feature working; every SKIP documents its precondition; every gate has a paired mutation in meta_test_*.sh proving the gate isn't itself a bluff. The covenant is restated verbatim in Constitution.md, CLAUDE.md, AGENTS.md, and QWEN.md (per the 2026-05-21 operator mandate) so that any tool which doesn't expand @imports still reads it. The upstream source lives in constitution/Constitution.md §11.4.

CodeGraph (code-intelligence)

This project is wired with CodeGraph per constitution/Constitution.md §11.4.78. The MCP server is configured for Claude Code, OpenCode, Kimi CLI, Crush, and Qwen Code — see docs/codegraph/README.md for the install + per-agent wiring contract.

License

Apache 2.0 — see LICENSE.